

プログラミング入門2

第11回 便利な構文、ファイル処理

篠埜 功

今日の内容

- 便利な構文を紹介
 - コンマ演算子
 - 増分演算子++、減分演算子--(それぞれ前置と後置がある)
- ファイル処理

コンマ演算子

for文の括弧内など、式が1つしか書けないところに2つ以上の式を書きたい場合に、コンマ演算子を用いて1つの式にする。

コンマ演算子

コンマ演算子を使った式の構文

式, 式

式をコンマで繋いで得られたものも式である。
よって式を3つ以上コンマで区切ったものも式である。(コンマ演算子は左結合)

式 e_1 , e_2 の意味

まず式 e_1 を評価し、次に e_2 を評価する。式 e_1, e_2 の値は、式 e_2 の評価結果である。

(補足)つまり、 e_1 の評価結果は捨てられるので、 e_1 に副作用(代入など)がないとこの構文を使う意味がない。

式 e_1 , e_2 の型

式 e_1, e_2 の型は式 e_2 の型である。

例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int a, i, j;
    a = (i=3, j=4);
    printf ("a=%d, i=%d, j=%d¥n", a, i, j);
    return 0;
}
```

赤字の部分がコンマ演算子を使った式である。
赤字の式の値は、式j=4の値、すなわち4である。
これがaに代入されるので、aの値は4となる。

for文に入れた例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int i, j;
    for (i=0, j=0; i<4; i=i+1, j=j+1)
        printf ("i=%d, j=%d¥n", i, j);
    return 0;
}
```

赤字の部分が、コンマ演算子を使った式の例である。

増分演算子(前置)

前置増分演算子を使った式の構文

++式

式は、アドレスを持ち、かつ値が変更可能(つまり代入式の左辺に書ける式)でなければならない。あと、式の型は、1との足し算ができる型でなければならない。

式 ++e の意味

eが一度だけ評価されるという点以外、代入式 $e=e+1$ と同じ意味である。

式 ++e の型

式 ++e の型は式 e の型である。

減分演算子--も同様に定義される。

典型例

```
#include <stdio.h>
int main (void) {
    int i, a[5]={1,2,3,4,5};
    for (i=0; i<5; i=i+1)
        printf ("a[%d]=%d¥n", i, a[i]);
    return 0;
}
```

```
#include <stdio.h>
int main (void) {
    int i, a[5]={1,2,3,4,5};
    for (i=0; i<5; ++i)
        printf ("a[%d]=%d¥n", i, a[i]);
    return 0;
}
```

for文においてよく使われる。i=i+1の代りに++i あるいは i++ (後述)と書くと、キーボードを打つ回数が若干減るので便利。(この例では式++iの値は使われないので、++iでもi++でも同じ。)

(参考) ++e と e=e+1 が異なる場合

```
#include<stdio.h>
int main (void) {
    int a[5]={10,20,30,40,50};
    int *p, i;
    p=a;
    ++(*(++p));
    for (i=0; i<5; i++)
        printf ("a[%d]=%d¥n", i, a[i]);
    return 0;
}
```

赤字の部分がe

```
#include<stdio.h>
int main (void) {
    int a[5]={10,20,30,40,50};
    int *p, i;
    p=a;
    *(++p) = *(++p) + 1;
    for (i=0; i<5; i++)
        printf ("a[%d]=%d¥n", i, a[i]);
    return 0;
}
```

赤字の部分がe

左のプログラムでは、ポインタpの値は1回だけ1が足されるが、右のプログラムでは2回、1が足される。

(注意)代入式において、左辺と右辺のどちらを先に評価するかは未規定である。したがって、*(++p)=*(++p)+1のように、左辺、右辺に関連のある副作用のある式を書くのは避けるべき。++(*(++p)) については、意味は一意である。

増分演算子(後置)

後置増分演算子を使った式の構文

式`++`

式は、アドレスを持ち、かつ値が変更可能(つまり代入式の左辺に書ける式)でなければならない。あと、式の型は、1との足し算ができる型でなければならない。

式 `e++` の意味

`e++`式の値は1を足す前の`e`の値であり、その後1が足される。

式 `e++` の型

式 `e++` の型は式`e`の型である。

減分演算子`--`も同様に定義される。

ファイル処理

これまでファイルの操作はemacsあるいはcp, mvなどのコマンドで行っていたが、C言語のプログラムでファイルを操作することができる。ファイルを操作するためのライブラリ関数が提供されている。

fopen --- ファイルを開く(ファイルをプログラムから扱えるように準備する)

fclose --- ファイルを閉じる(ファイルを扱える状態においては、プロセスにおけるファイル用の表のエントリを1つ分占めている。それが解放される。)

fprintf --- ファイルへの書き込み

fscanf --- ファイルからの読み取り

などのライブラリがある。

これらのライブラリ関数を使う場合は、stdio.hをインクルードする。(printfを使う場合と同じ)

例（入力して確認）

```
#include <stdio.h>
int main (void) {
    FILE *fp;
    fp = fopen ("test", "r");
    if (fp==NULL) {
        printf ("オープン失敗¥n");
        return 0;
    }
    printf ("ファイルをオープンしました¥n");
    fclose (fp);
    printf ("ファイルをクローズしました¥n");
    return 0;
}
```

testという名前のファイルをオープンしてクローズするだけのプログラム。

testという名前のファイルがない場合には「オープン失敗」と出力して終了。

testという名前のファイルを自分で作ってから実行してください。

FILE型

ライブラリ関数fopen, fprintf, fscanf, fcloseにおいては、FILE型のオブジェクトを介してファイルへのアクセスを行う。FILE型のオブジェクトにファイルへのアクセスに必要な情報が格納されている。FILE型の具体的なデータ構造は処理系によって異なる。

ライブラリ関数fopen

fopenは、ファイル名とモードを引数にとり、FILE型へのポインタを返り値として返す。オープンに失敗した場合はNULLポインタを返す。

[モード]

r --- 読み取りモードでオープン

w --- 指定されたファイルがない場合は、書き込みモードでファイルを新たに生成してオープン。ある場合は、ファイルをオープンして、既存の内容を全部消す。

他にもいくつかモードがある。詳しくはman fopenで確認。

ライブラリ関数fclose

fcloseは、FILE型へのポインタを引数として受け取り、そのファイルを閉じる。

ライブラリ関数fprintf

第1引数にFILE型へのポインタを受け取り、そこへ書き込む。第2引数以降はprintfと同じ形式である。

printf関数は、fprintf関数の第一引数にstdoutを指定した場合と同じ意味である。(stdoutは標準出力を表すFILE型へのポインタ。)

例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    FILE *fp;
    fp = fopen ("test", "w");
    if (fp==NULL) {
        printf ("ファイルをオープンできません\n");
        return 0;
    }
    fprintf (fp, "%d+%d=%d\n", 1, 1, 2);
    fclose (fp);
    return 0;
}
```

testというファイル
に1+1=2と書きこむ
プログラム。

testというファイル
があったら、その内
容は消されてから
1+1=2と書きこまれ
る。

testというファイル
がなければ、新た
に作成されてから
1+1=2と書きこまれ
る。

fopenが失敗する場合

fopen (“test”, “w”)が失敗するのは、testというファイルが存在し、かつ(自分に)書き込み権限がない場合。例えば、

```
% chmod 444 test
```

とすると、testというファイルへの書き込みができなくなる。この状況でさきほどのプログラムを実行すると

```
% ./a.out
```

ファイルをオープンできません

と出力される。もし

```
% chmod 000 test
```

とすると、読み込み権限もなくなる。この状況下では、fopen (“test”, “r”)も失敗する。元に戻すには、

```
% chmod 644 test
```

のようにすればよく、(自分が)ファイルの読み書きができる状態になる。

ライブラリ関数fscanf

第1引数にFILE型へのポインタを受け取り、そこから読み取る。第2引数以降はscanfと同じ形式である。

scanf関数は、fscanf関数の第一引数にstdinを指定した場合と同じ意味である。(stdinは標準入力を表すFILE型へのポインタ。)

[少し詳しい説明]

fscanfは、第2引数によって指定されるフォーマットに従ってファイルから入力を読み取って変換し、第3引数以降に受け取ったポインタの指す先に(変換指定での照合が成功したら)順次代入する。変換指定と合わなかった時点で読み取りが終了する(合わなかった部分以降はストリーム上に残る。詳細はman scanfでマニュアルを参照)。変換が成功した個数が返り値として返される。

例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    FILE * fp;
    int num=0;
    char name [100];
    double height, weight;
    double hsum=0.0, wsum=0.0;
    fp = fopen ("data", "r");
    if (fp==NULL) {
        printf ("オープン失敗¥n");
        return 0;
    }
```

Taro	160.0	59.3
Jiro	162.0	51.6
Saburo	182.0	76.5
Shiro	170.0	60.7

```
/* 続き*/
while (fscanf (fp, "%s%lf%lf", name,
                &height, &weight) == 3) {
    num++;
    hsum = hsum + height;
    wsum = wsum + weight;
}
printf ("平均身長: %5.1fcm¥n", hsum / num);
printf ("平均体重: %5.1fkg¥n", wsum / num);
fclose (fp);
return 0;
}
```

左のような内容のファイルをdataという名前で講義用web pageに置いています。これをダウンロードしてから上記プログラムを実行してください。

補足

さきほどのプログラムにおいて、名前の情報は使われないので、標準入力のうち、名前の部分をnameに書き込む必要がない(配列nameの領域が無駄になっている)。

```
fscanf (fp, "%s%lf%lf", name, &height, &weight)
```

の部分を

```
fscanf (fp, "%*s%lf%lf", &height, &weight)
```

とすると、名前の部分を読み飛ばすことができるので、このように書いても良い。

補足

[fscanfの返り値について]

変換がまだ1つも行われていないときにファイルの終端に来た場合、fscanfの返り値がEOFになる。この演習室だと-1。変換が1つ以上行われた後で途中でファイルの終端に来た場合は、fscanfの返り値が、変換に成功した個数(1以上の値)となる。fscanfの返り値が0になるのは、最初の変換が失敗した場合。例えば、

```
int x;
```

```
fscanf(stdin, "%d", &x);
```

で数字以外(例えばaなど)が入力だった場合、fscanfの返り値が0になる。

基本課題1

さきほどのプログラムは、身長、体重データをファイルから読み取って平均値を表示するプログラムであった。同じ形式のファイル(ファイル名はdataとする)からデータを読み取り、身長、体重の最大値を以下の例のように表示するプログラムを作成せよ。

[さきほどのデータでの実行例]

身長の最大値: 182.0cm

体重の最大値: 76.5kg

身長、体重のprintfでの表示は、変換指定子として%5.1fを使ってください。

基本課題2

キーボードからファイル名 f および文字列 s を読み取り、文字列 s をファイル f に書きこむプログラムを書け。文字列 s は英語のアルファベットのみで、空白等を含まないと仮定してよい。(ファイルに書き込むときに文字列の後ろに改行を入れないようにしてください)

[実行例]

```
$ ./a.out
```

文字列をファイルに書き込みます

ファイル名を入力してください: **fff**

文字列を入力してください: **abcde**

```
$ cat fff
```

```
abcde
```

存在するファイルの名前を打ち込まないように注意してください。

発展課題1

掛け算の九九の表をファイルに書き込むプログラムを作成せよ。ファイル名はkuku.txtとする。形式は以下のよう
にせよ。(空白の個数を調整し、表示位置がそろうように
せよ。printfの変換指定子として%3dを用いればよい。81
の後に改行を入れてください。)

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45
6	12	18	24	30	36	42	48	54
7	14	21	28	35	42	49	56	63
8	16	24	32	40	48	56	64	72
9	18	27	36	45	54	63	72	81

(注意)九九のかけ算を
プログラム中で行って
ください(手で81個打ち
込まないようにしてく
ださい)。二重ループ
で書いてください。

発展課題2

基本課題1と同じ形式のファイル(ファイル名はdataとする)を読み込み、さらにキーボードから名前を入力させ、その名前の人のデータを以下の実行例のように画面に表示するプログラムを書け。

[さきほどのデータでの実行例]

名前を入力: Jiro

Jiro 162.0cm 51.6kg

(ヒント) 文字列の比較は、strcmpというライブラリ関数を使って行ってください。もちろん自分で書いてもいいですが。ライブラリ関数strcmpを使うときは、string.hをincludeしてください。使い方は、manコマンドで調べてください。printfの変換指定子は%.1fを使ってください。

発展課題3

基本課題1と同じ形式のファイル(ファイル名はdataとする)を、名前、体重、身長順に並べ替えてdata2というファイルに書き出すプログラムを書け。
出力ファイルは以下のヒントの形式に従ってください。

(ヒント) fprintfの出力形式の例(各行):

```
"%-6s %3.1f %3.1f¥n"
```

%-6sにおけるマイナスは左詰めを表す。マイナスがない場合は右詰めを表す。6は最小フィールド幅(表示する最小の幅)を表す。%3.1fは、全体の幅(小数点を含む)が最小で3かつ小数点以下を1桁表示する。

発展課題4

以下のような形式の、商品番号、商品名、値段に関するファイル(ファイル名はgoodsとする)を読み込んでそのまま表示するプログラム(shopping.c)を講義用web pageに置いてある。これを参考に、右下のように商品リストを表示した後、キーボードから商品番号と個数を1組入力させ、合計金額を表示するプログラムを書け。商品リスト中の金額と合計欄の個数、金額は3桁ごとにコンマを入れて表示せよ。goodsの商品数は50以内であることを前提として良い。商品を表すデータ型として左下の構造体を用いよ。講義用web pageにgoodsのファイルの例を置いてある。整数型のoverflowは対処しなくてよいものとする。

```
39065 ポテトチップス 148
75990 メロン 1280
... (略) ...
```

```
typedef struct{
    int id;
    char name[100];
    int price;
} goods;
```

[実行例]

\$./a.out

買い物をします

--商品リスト--

39065 ポテトチップス 148円

75990 メロン 1,280円

... (略) ...

商品番号と個数を入力

商品番号: 75990

個数: 30000

メロンが30,000個で合計38,400,000円です。

0から9までの数から文字への変換について(発展課題4)

- 0から9までの数がint型の変数xに入っている場合、 $x + '0'$ が変数xに格納されている数に対応する数字を表す数(を計算する式)である。
- 文字0,1,2,3,4,5,6,7,8,9を表す数は左隣の文字を表す数+1とISO規格で定められている。

参考課題1

キーボードからファイル名fおよびint型の数nを入力させ、nをファイルfに書きこむプログラムを書け。

[実行例]

```
$ ./a.out
```

int型の数をファイルに書き込みます

ファイル名を入力してください: fff

int型の数を入力してください: 300

```
$ cat fff
```

```
300
```

```
$
```

参考課題1

解答例

```
#include<stdio.h>
int main(void)
{
    FILE *fp;
    char fileName[30];
    int n;
    printf("int型の数をファイルに書き込みます¥n");
    printf("ファイル名を入力してください: ");
    scanf("%s", fileName);
    fp = fopen(fileName, "w");
    if (fp==NULL) {
        printf ("オープン失敗¥n");
        return 0;
    }
    printf("int型の数を入力してください: ");
    scanf("%d", &n);
    fprintf(fp,"%d¥n", n);
    fclose(fp);
    return 0;
}
```

本演習で扱わなかった内容

- ビット演算(教科書 p.164-171)
- 関数へのポインタ(ポインタの極意 第9章)
- switch文(教科書 p.54-57)
- do while文(教科書 p.60-67)
- マクロ(教科書 p.96-97)
- 不完全型(実践編 p.27)
- 変数の記憶域期間(教科書 p.142-145)
- const型修飾子(教科書 p.133)
- 複合代入演算子(教科書 p. 66)

これらについては、各自上記の教科書、参考書を参照してください。最終的には規格書を参照することになります。